



6강. NumPy 벡터 연산 — ndarray·슬라이싱·브로드캐스팅



학습 목표

- ndarray가 Python list와 메모리 구조상 어떻게 다른지, 왜 벡터화 연산이 수십 배 빠른지 설명할 수 있습니다.
- shape·dtype·axis 개념을 이해하고 다차원 배열을 자유롭게 다룰 수 있습니다.
- 슬라이싱·불리언 마스크·팬시 인덱싱으로 원하는 데이터를 추출할 수 있습니다.
- 브로드캐스팅 규칙을 이해하고 모양이 다른 배열 간 연산을 설계할 수 있습니다.
- 반복문 코드를 벡터화 코드로 바꿔 로봇 데이터 처리를 가속할 수 있습니다.



목차

들어가며

1. ndarray는 왜 빠를까요? — 메모리 구조

2. 기본기 — 생성, shape, dtype, axis

3. 슬라이싱 — 복사 없이 잘라 보기

 불리언 마스크 — 조건으로 골라내기

4. 브로드캐스팅 — 모양이 달라도 연산하기

5. 벡터화 사고방식 — 반복문을 지우는 연습

미니 퀴즈

실습

정리

들어가며

LiDAR 한 바퀴 스캔은 수천 개의 거리값, 카메라 한 프레임은 수백만 개의 픽셀입니다. 로봇의 인지 계층(2강)은 이런 대량 숫자 데이터를 **매 주기** 처리해야 합니다. 30Hz 카메라라면 33ms 안에 수백만 픽셀 연산을 끝내야 하지요.

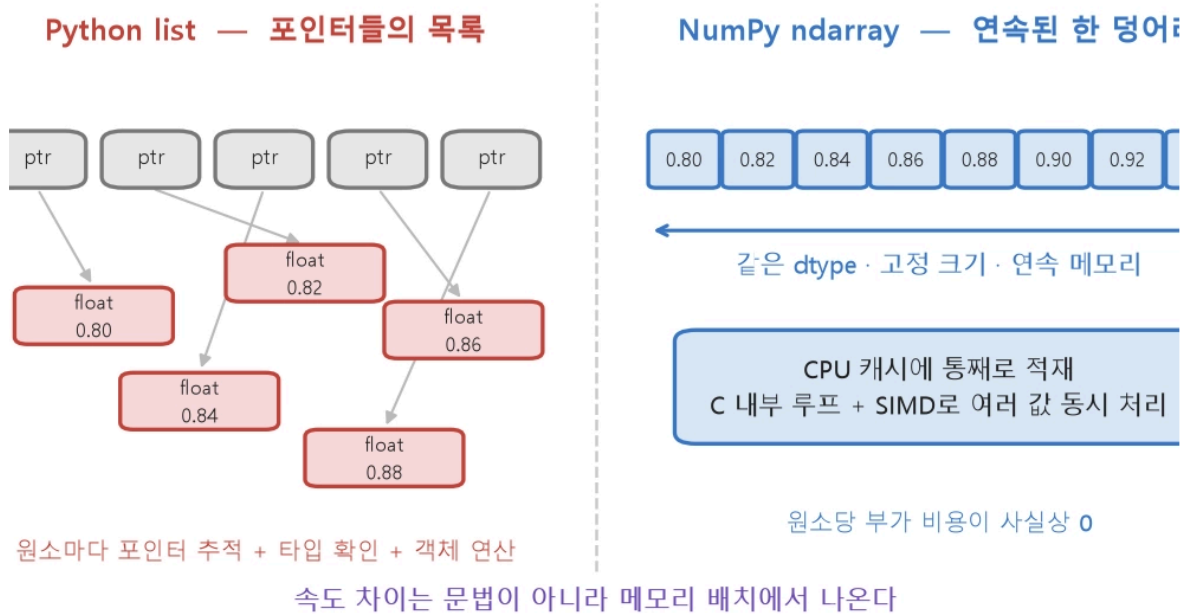
순수 Python 반복문으로는 어림도 없습니다. 이때 등장하는 것이 **NumPy** — Python 과 학계산의 토대이자, OpenCV·PyTorch·ROS2 메시지 처리의 공용 데이터 구조입니다. 핵심은 단 하나의 사상입니다: **"반복문을 쓰지 말고, 배열 전체를 한 번에 연산하라(벡터화)."** 이번 강의에서 그 원리와 도구를 익힙니다.

💡 **실습 환경:** 이번 강의부터는 5강에서 만든 venv 위의 **Jupyter Notebook**에서 진행합니다. 배열의 shape가 어떻게 바뀌는지, 마스킹 결과가 어떤 모양인지를 셀마다 즉시 확인하면서 읽어 나가세요. `%timeit` 으로 본문의 속도 비교를 직접 재 볼 수도 있습니다.

1. ndarray는 왜 빠를까요? — 메모리 구조

NumPy의 중심 객체는 **ndarray**(n-dimensional array)입니다. list와의 차이는 문법이 아니라 **메모리 구조**에 있습니다.

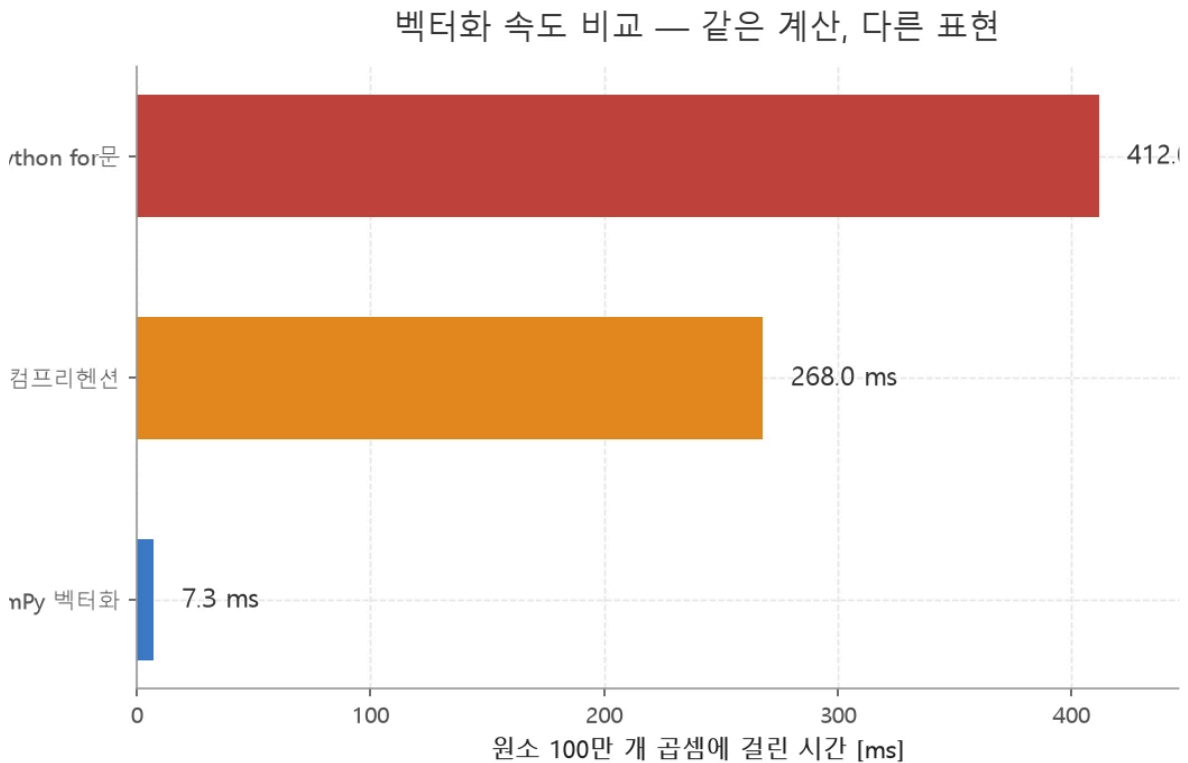
메모리 구조 — Python list vs NumPy ndarray



images/06_메모리_구조.png — 메모리 구조: list vs ndarray

💡 **해석:** (왼쪽) Python list는 **포인터들의 목록**입니다. 각 원소는 메모리 여기저기 흩어진 별도 객체라서, 반복문이 원소마다 "포인터 따라가기 + 타입 확인 + 객체 연산"을 반복합니다. (오른쪽) ndarray는 **같은 타입(dtype)의 값이 연속된 한 덩어리**입니다. CPU 캐시에 착착 올라가고, C로 구현된 내부 루프가 SIMD 명령으로 여러 값을 한꺼번에 처리합니다.

그 차이가 실제로 얼마나 될까요? 원소 100만 개짜리 두 배열의 곱셈을 실측한 결과입니다.



NumPy 벡터화가 반복문 대비 약 56배 — 30Hz(33ms) 마감을 지키느냐 놓치느냐의 차이

images/06_벡터화_속도.png — 벡터화 속도 비교

💡 **해석:** 같은 계산이 Python 반복문 대비 NumPy 벡터화에서 **수십 배**(이 실측에서는 약 56배) 빠릅니다. 30Hz 인지 루프처럼 마감이 있는 코드에서 이 차이는 "동작하는 시스템"과 "마감을 놓치는 시스템"의 차이입니다.

✔ **핵심 규칙:** ndarray를 만들어 놓고 Python for문으로 원소를 하나씩 돌면 NumPy의 이점이 전부 사라집니다. 배열 전체에 대한 연산(벡터화)으로 표현하는 것이 NumPy의 사용법입니다.

2. 기본기 — 생성, shape, dtype, axis

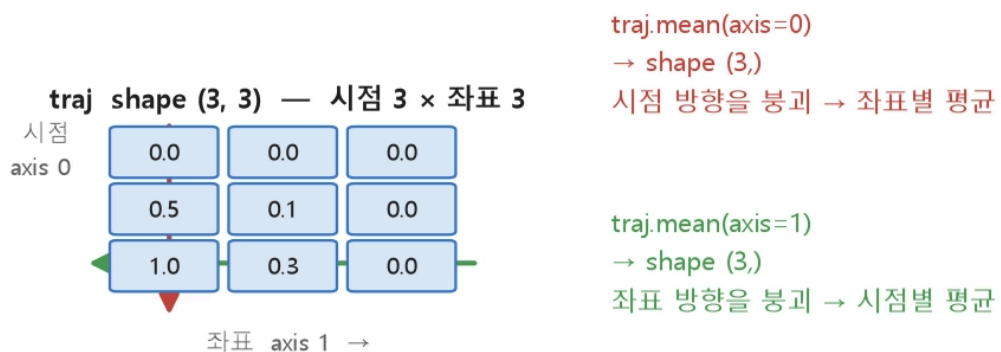
```
import numpy as np
a = np.array([0.8, 0.82, 0.85, 0.79]) # list → 1차원 배열
scan = np.zeros(360) # 0으로 채운 배열 (LiDAR 1회전)
angles = np.linspace(0, 2*np.pi, 360) # 0~2π 균등 분할
img = np.random.rand(480, 640, 3) # 480×640 RGB 임의의 이미지
a.shape # (4,) — 각 차원의 크기
img.shape # (480, 640, 3) — 행, 열, 채널
a.dtype # float64 — 원소 타입 (전부 동일!)
img.ndim # 3 — 차원 수
```

shape는 NumPy 사고의 중심입니다. 로봇 데이터의 전형적인 shape를 기억해 두세요.

데이터	shape	의미
LiDAR 1스캔	(360,)	각도별 거리
위치 궤적	(N, 3)	N개 시점 × (x, y, z)
회전 행렬	(3, 3)	3차원 회전 (모듈 ② TF2와 연결)
컬러 이미지	(H, W, 3)	높이 × 너비 × RGB

axis(축)는 "어느 방향으로 연산할지"입니다.

shape와 axis — 어느 차원을 붕괴시킬 것인가



axis=k 는 "k번째 차원을 붕괴시킨다"로 읽으면 헷갈리지 않는다

images/06_shape_axis.png — shape와 axis

💡 **해석:** (3, 3) 궤적 배열에서 `axis=0` 으로 평균을 내면 **시점 방향이 붕괴**되어 좌표별 평균 (3,) 이 남고, `axis=1` 이면 **좌표 방향이 붕괴**되어 시점별 평균이 남습니다. 결과의 shape를 먼저 떠올리는 습관이 다차원 배열 사고의 핵심입니다.

(N, 3) 궤적에서 코드로 확인하면:

```
traj = np.array([[0.0, 0.0, 0.0], [0.5, 0.1, 0.0], [1.0, 0.3, 0.0]]) # shape (3, 3): 시점 3개 × 좌표 3
traj.mean(axis=0) # 시점 방향으로 평균 → 좌표별 평균 위치, shape (3,)
traj.mean(axis=1) # 좌표 방향으로 평균 → 시점별 평균(의미 없음 예시), shape (3,)
```

💡 **axis=k는 "k번째 차원을 붕괴시킨다"** 로 읽으면 헷갈리지 않습니다. (3, 3) 에서 `axis=0` 을 붕괴 → (3,).

dtype도 실무에서 중요합니다.

dtype — uint8 오버플로라는 고전적 함정

```
img = np.array([250], dtype=np.uint8)
img + 10
→ array([4], dtype=uint8)
```

255를 넘는 순간 0으로 되감김

```
img.astype(np.float64) + 10
→ array([260.0])
```

계산 전 승격이 관례

dtype	표현 범위	크기	쓰임
uint8	0 ~ 255	1 B	이미지 픽셀
float32	약 $\pm 3.4e38$	4 B	GPU·딥러닝 텐서
float64	약 $\pm 1.8e308$	8 B	좌표·회전 계산 (NumPy 기본)

shape만큼이나 dtype을 의심하라 — 값이 이상하면 절반은 dtype 문제

images/06_dtype_오버플로.png — dtype와 오버플로

💡 **해석:** 이미지 픽셀은 보통 `uint8` (0~255, 1바이트), 좌표 계산은 `float64` (8바이트)입니다. `uint8` 끼리 더하면 255를 넘는 순간 **오버플로**가 나서 250+10이 4가 됩니다. 계산 전 `img.astype(np.float64)` 로 승격하는 것이 관례입니다. **shape만큼이나 dtype을 의심하라** — 값이 이상할 때 절반은 dtype 문제입니다.

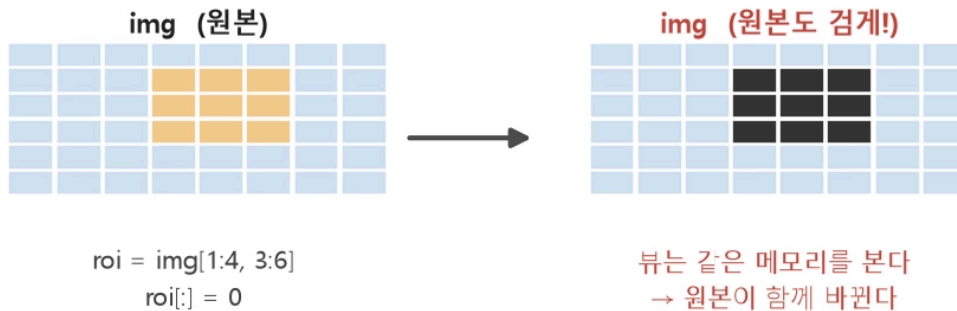
3. 슬라이싱 — 복사 없이 잘라 보기

리스트처럼 `[시작:끝:간격]` 으로 자르되, **차원마다 콤마로** 지정합니다.

```
scan = np.random.rand(360) * 10 # LiDAR 스캔 (0~10m) scan[0:90] #  
전방 90° 구간 scan[:,2] # 한 칸 걸러 다운샘플링 scan[-10:] # 마지막 10개  
img = np.random.rand(480, 640, 3) img[100:200, 300:400] # 이미지의  
관심 영역(ROI) 잘라내기 img[:, :, 0] # R 채널만 → shape (480, 640)  
img[:,2, ::2] # 가로세로 절반 해상도로 다운샘플
```

여기서 NumPy의 중요한 성질 — **슬라이스는 복사가 아니라 뷰(view)** 입니다.

슬라이스는 뷰(view), 마스킹·팬시 인덱싱은 복사(copy)



복사가 필요하면 `.copy()` | 슬라이스 = 뷰 · 불리언 마스킹 / 팬시 인덱싱 = 복사

images/06_슬라이싱_뷰.png — 슬라이스는 뷰

💡 **해석:** 슬라이스로 잘라낸 ROI는 **원본과 같은 메모리**를 가리킵니다. ROI를 검게 칠하면 원본 이미지의 그 영역도 검게 변합니다. 대용량 데이터를 복사 없이 다루게 해 주는 강력한 기능이지만, 모르고 쓰면 "원본이 왜 바뀌지?" 버그가 됩니다.

```
roi = img[100:200, 300:400] roi[:] = 0 # ROI를 검게 칠하면... # img의 해당 영역도 검게 변합니다! 같은 메모리를 보고 있기 때문
safe = img[100:200, 300:400].copy() # 독립 사본이 필요하면 copy()
```

5강의 `b = a` 참조 문제와 같은 계열의 함정입니다. 노트북에서는 셀을 다시 실행할 때마다 원본이 계속 오염되어 혼란이 커지므로, **의심스러우면 `.copy()`가 안전한 습관**입니다.

불리언 마스킹 — 조건으로 골라내기

벡터화의 꽃입니다. 조건식이 **True/False 배열(마스크)**을 만들고, 그 마스크로 원소를 선택합니다.

images/06_마스킹.png — 불리언 마스킹

💡 **해석:** `scan < 1.0` 이라는 조건식 하나가 원본과 같은 길이의 **True/False 배열**을 만들고, 그 배열을 인덱스 자리에 넣으면 True인 원소만 남습니다. **조건 → 마스크 → 선택**이라는 세 박자가 벡터화 사고의 기본형입니다.

```
scan = np.array([0.3, 8.2, 0.5, 12.0, 0.45, 9.9]) mask = scan < 1.0 # array([True, False, True, False, True, False])
near = scan[mask] # array([0.3, 0.5, 0.45]) - 1m 이내 장애물만
scan[scan > 10.0] = 10.0 # 10m 초과 값을 10으로 클리핑 (센서 최대거리 처리) # 복합 조건: & (and), | (or) - 괄호 필수!
valid = scan[(scan > 0.1) & (scan < 10.0)] # 유효 측정만
```

반복문으로 쓰면 5줄+조건분기가 될 코드가 한 줄이 되고, 속도는 수십 배 빠릅니다. LiDAR 노이즈 제거, 이미지 임계값 처리(픽셀 밝기 > 128인 곳만) 등 인지 코드의 표준 관용구입니다.

▶ **심화** — 팬시 인덱싱과 `np.where`

4. 브로드캐스팅 — 모양이 달라도 연산하기

shape가 같은 배열끼리는 원소별 연산이 자명합니다. NumPy는 여기서 한 발 더 나가, **모양이 다른 배열 간의 연산 규칙**을 제공합니다.

브로드캐스팅 — 모양이 달라도, 메모리 복사 없이

A (3, 3)				b (3,)				결과 (3, 3)		
1	2	3	+	10	20	30	=	11	22	33
4	5	6		10	20	30		14	25	36
7	8	9		10	20	30		17	28	39

각 행에 자동으로 "늘려짐"
실제 메모리 복사는 없음

규칙 — shape를 뒤에서부터 짝지어, ① 같거나 ② 한쪽이 1이면 호환 (1인 쪽이 늘어남)

images/06_브로드캐스팅.png — 브로드캐스팅

💡 **해석:** (3, 3) 행렬에 (3,) 벡터를 더하면, 벡터가 각 행에 **자동으로 "늘려져"** 적용됩니다. 중요한 것은 이 늘림이 **실제 메모리 복사 없이** 일어난다는 점 — 큰 배열을 만들지 않고도 같은 효과를 냅니다.

규칙은 두 줄입니다. 두 배열의 shape를 **뒤에서부터** 짝지어 비교해서:

1. 두 크기가 **같거나**,
2. **한쪽이 1이면** → 호환 (1인 쪽이 상대 크기로 늘어남)

모든 짝이 호환이면 연산 가능, 아니면 에러입니다.

```
A = np.random.rand(480, 640, 3) # 이미지 m = np.array([0.5, 1.0, 2.0]) # 채널별 계수, shape (3,) A * m # A: (480, 640, 3) # m: ( 3) ← 뒤에서부터: 3=3 ✓, 나머지는 m에 차원이 없으므로 자동 확장 # 결과: (480, 640, 3) — R×0.5, G×1.0, B×2.0 이 한 줄로!
```

로봇에서 자주 쓰는 패턴들:


```
# ① 궤적 전체를 원점 이동: (N,3) - (3,) traj_centered = traj - traj[0]
# ② 궤적 전체 회전: (3,3) 행렬 @ (N,3) → 행렬곱 + 전치 관용구 R = np.array([[0, -1, 0], [1, 0, 0], [0, 0, 1]]) # z축 90° 회전
traj_rot = traj @ R.T # (N,3) - N개의 점을 한 번에 회전! # ③ 스칼라도 브로드캐스팅: 모든 거리값을 mm → m scan_m = scan_mm * 0.001
```

②번이 특히 중요합니다 — 모듈 ②의 TF2 좌표 변환, Level 2의 기구학이 전부 "점들의 배열 × 회전 행렬"이며, NumPy에서는 반복문 없이 @ 하나로 끝납니다.

▶ **심화** — 차원 추가(newaxis)로 브로드캐스팅 설계하기

5. 벡터화 사고방식 — 반복문을 지우는 연습

마지막으로, 반복문 코드를 벡터화로 바꾸는 사고 과정을 봅니다. "이동 로봇의 속도 명령이 안전 한도를 넘는 비율"을 계산한다고 합시다.

```
# ✗ 반복문 사고 count = 0 for v in velocity_log: if abs(v) > 1.0:
count += 1 ratio = count / len(velocity_log) # ✓ 벡터화 사고: "조건 → 마스크 → 집계" ratio = (np.abs(velocity_log) > 1.0).mean() #
True=1, False=0의 평균 = 비율
```

벡터화 전환의 일반 공식:

반복문 패턴	벡터화 대응
원소마다 계산	배열 전체 연산 (<code>a*2</code> , <code>np.sin(a)</code>)
if로 걸러서 모으기	불리언 마스크 (<code>a[cond]</code>)
if로 값 바꾸기	<code>np.where(cond, x, y)</code> 또는 <code>a[cond] = x</code>
합·평균·최대 구하기	<code>sum()</code> , <code>mean()</code> , <code>max()</code> (+ axis)
두 배열 짝지어 계산	같은 shape 연산 또는 브로드캐스팅
최솟값의 "위치" 찾기	<code>np.argmin</code> , <code>np.argmax</code>

이 표만 체득해도 인지·제어 데이터 처리 코드의 90%를 벡터화할 수 있습니다. 그리고 이 감각은 모듈 ④의 제어 시뮬레이션(시간축 배열 연산)과 이후 PyTorch(문법이 NumPy와 거의 동일)로 그대로 이어집니다.

미니 퀴즈

💡 아래 문항을 먼저 스스로 풀어 본 뒤 실습으로 넘어가세요. 답안은 실습 결과물과 함께 제출합니다. (총 10점)

1. (2점) ndarray 가 Python list 보다 빠른 근본 이유는?

- ① C로 작성되어 있어서
- ② 같은 dtype 의 값이 연속된 메모리에 놓여 캐시와 SIMD를 활용할 수 있어서
- ③ 메모리를 적게 써서
- ④ 반복문이 자동으로 병렬화되어서

2. (2점) `traj` 의 shape 가 `(100, 3)` 일 때 `traj.mean(axis=0)` 과 `traj.mean(axis=1)` 의 결과 shape 를 각각 쓰고, 각각 무엇을 평균인지 설명하세요.

3. (2점) 다음 코드 실행 후 `img` 의 해당 영역은 어떻게 되는지와 그 이유를 쓰세요.

```
roi = img[100:200, 300:400] roi[:] = 0
```

4. (2점) shape `(480, 640, 3)` 배열과 shape `(3,)` 배열을 곱할 수 있는 이유를 브로드캐스팅 규칙으로 설명하세요.

5. (2점) `uint8` 이미지에서 `250 + 10` 이 4가 되는 이유와, 올바른 처리 방법을 쓰세요.

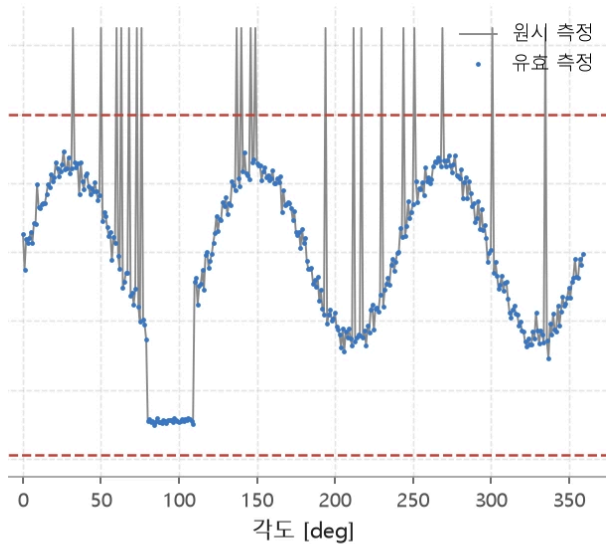
실습

노트북 `notebooks/06_numpy.ipynb` 에서, 가상의 LiDAR 데이터로 **장애물 감지 파이프라인**을 벡터화로 구현해 보겠습니다.

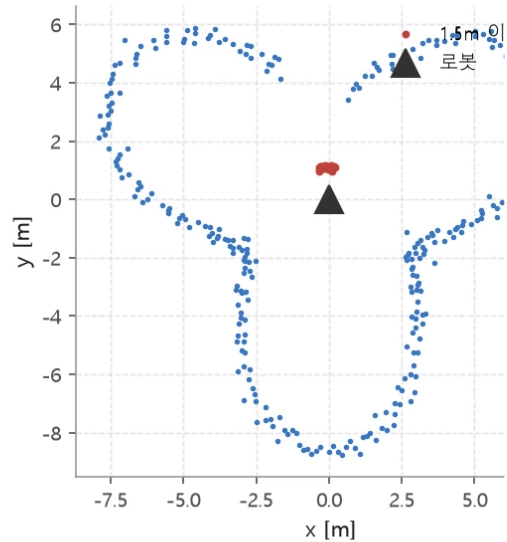
```
import numpy as np rng = np.random.default_rng(0) # 시드 고정 - 재현
가능한 실습 scan = rng.random(360) * 12 # 0~12m 임의 거리 angles =
np.linspace(0, 2*np.pi, 360) # 각 측정의 각도 [rad]
```

완성하면 다음과 같은 그림이 나옵니다.

① 극좌표 원시 스캔 + 마스킹



② $x = r \cdot \cos\theta$, $y = r \cdot \sin\theta$ (반복문



세 단계 전부 for문 한 줄 없이 배열 연산만으로 표현된다

images/06_lidar_극좌표.png — LiDAR 벡터화 파이프라인

💡 **해석:** (왼쪽) 극좌표로 들어온 원시 스캔에서 유효 범위(0.1~10m) 밖을 마스킹으로 걸러내고, (오른쪽) $x = r \cdot \cos\theta$, $y = r \cdot \sin\theta$ 로 직교좌표로 옮겨 위험 구간을 표시했습니다. 세 단계 전부 for문 없이 배열 연산만으로 표현됩니다.

1. **전처리:** 0.1m 미만(노이즈)과 10m 초과(최대거리)를 제외한 **유효 측정**만 남기세요. 유효 비율도 계산해 보세요. (마스킹)
2. **최근접 장애물:** 유효 측정 중 최솟값과 그 각도(도 단위)를 구하세요. (`argmin`, 단 무효값 처리에 주의)
3. **직교좌표 변환:** 모든 유효 측정을 $x = r \cdot \cos(\theta)$, $y = r \cdot \sin(\theta)$ 로 변환해 (`N`, `2`) 배열을 만드세요. (반복문 금지!)